Hangman Web Application — Development Report

CSE 4095 Individual Assignment · Priyanshi Sitlani · GitHub: https://github.com/pr1yanshii/hangman-web · Live: https://hangman-web-0je9.onrender.com

1. What I Built

For this assignment, I built a Hangman game that you play in a web browser. It is a smaller version of the course specification. As planned in my *TODO.md*, I left out the terminal client, bots, evaluation mode, scoring, leaderboard, and replay system. The project has three parts:

- **The web page** (*static/* folder): HTML, CSS, and plain JavaScript. This is what the player sees and clicks.
- **The Flask server** (*server.py*): it receives requests from the page and sends back JSON. It keeps each game in memory under its own random ID.
- **The game logic** (*game.py* and *words.py*): plain Python with no Flask code, so I could test the rules on their own.

The most important rule is that the server is in charge. The browser never gets the hidden word until the game is over.

2. The Game Logic and the API

The *HangmanGame* class remembers every guess and which ones were wrong. It shows the word as blanks, like `_ i _ _ _ _`, and fills in letters as they are guessed. You win when every letter is showing, and you lose after six wrong guesses. A guess that is not a single letter, was already guessed, or comes after the game ended is rejected, and nothing changes. The word is only added to the game data once the game is over, so it can't slip out during play. Each difficulty (Easy, Medium, Hard) has its own list of 55 words.

The API has three endpoints: `POST /api/games` starts a game (201), `GET /api/games/<id>` returns the current game (200), and `POST /api/games/<id>/guess` sends a letter (200). When something goes wrong, the server sends back a JSON error message with a status code: 400 for a bad request or a repeated guess, 404 if the game doesn't exist, 405 for the wrong method, and 409 if the game is already over.

3. The Web Page

The page has a difficulty menu, a New Game button, the word as letter tiles, the number of tries left, lists of correct and wrong guesses, a hangman drawing, and an A–Z keyboard. You can click the letters or type them on your own keyboard. The drawing has six parts, and one more appears with each wrong guess.

This is how the page talks to the server. When you press New Game, *app.js* uses *fetch()* to send a POST request to `/api/games` with the difficulty you picked, and the server replies with JSON describing the new game. When you guess a letter, the page sends another POST request with that letter and redraws itself from the reply. The page doesn't decide anything by itself; it just shows what the server sends back. That is also why New Game works without reloading the page. If the server can't be reached, or the game no longer exists (for example after a server restart), the page shows a message instead of just breaking. On phones, the layout switches to one column with a smaller keyboard.

4. Testing and Deployment

- **Automated tests:** 103 pytest tests pass: 17 for the word lists, 32 for the game logic, 41 for the API, and 13 integration tests. They check every error code, broken JSON, two games not mixing up, and that no response contains the word while the game is still going. The integration tests also play full games with the real word lists and start the app from a different folder.
- **Browser checks:** a script controlled Chrome in the background and ran 36 checks on a local copy of the site, and all of them passed. It tested both keyboards, the drawing, winning and losing, New Game, error messages, and phone and tablet screen sizes. The checklist for testing by hand is in *docs/manual_tests.md*.
- **Deployment:** the code is on GitHub, and Render runs the live site. Render installs the packages with `pip install -r requirements.txt` and starts the server with `gunicorn server:app --workers 1 --bind 0.0.0.0:$PORT`. A script I ran against the live HTTPS site passed all 12 API checks, including the error codes and the hidden-word check.

5. How I Used Claude Code

I used **Claude Code** (by Anthropic, with the Opus 5.5 model) inside VS Code. First, I had it turn the course *TODO.md* into seven phases, each with tasks, tests, and a clear meaning of "done". After that, I asked it to build one or two phases at a time and gave it rules to follow. Claude Code wrote the code, tests, and documentation, ran the tests, and told me the results. I read each report, answered its questions about the scope, and set up the GitHub repository and the Render service myself. It only made commits when I approved them. One rule I kept repeating was: only tick a TODO item after it has been checked, and never say a manual test passed unless someone actually did it.

6. Challenges I Ran Into

- **Deciding how strict to be.** The specification says repeated guesses can be "ignored or rejected." I decided to reject them with a 400 error, and to send 409 if someone guesses after the game is over. I listed both choices in the README so they don't surprise anyone.
- **Games disappearing.** The games live in the server's memory. Gunicorn can run several workers, and each one has its own memory, so a game made in one worker wouldn't exist in another. I solved this by running only one worker. I also learned that a Render restart, or the free plan going to sleep, wipes all games. So the page shows a message and lets you start a new game.
- **Problems on my own computer.** Port 5000 was already taken by the macOS AirPlay Receiver, so I ran the app on port 8000 instead. Halfway through building the web page, my disk filled up and everything stopped until I cleared some space. When I tested the live site with a Python script, I got an SSL certificate error even though curl worked fine. The problem was my Python install, not the site, and pointing the script to the system's certificate file fixed it.

- **Tests failing for the wrong reason.** One test makes sure the hidden word never appears in a response. But game IDs are random hex characters (a–f and 0–9), so a word like "cafe" could show up inside an ID by accident. The tests use "python" instead, since none of its letters can appear in an ID. Another browser check failed only because the CSS shows letters in capitals. The app was fine; the test needed fixing.
- **Understanding how the page uses the API.** While checking the final work, I found a TODO item saying the page uses all three endpoints. That wasn't true. The page only uses the two POST requests, because each reply already contains the full game, so it never needs GET. I unchecked the item and explained it in the README. In my demo script, I call GET myself from the Chrome DevTools console.

7. Three Prompts I Used (exact wording; [...] means I skipped some lines)

Prompt 1 — Planning the smaller scope

```
2. Revise TODO.md to match the reduced assignment scope.
4. Organize the work into clear development phases.
5. Include every required feature, test, and deliverable.
[...]
For every phase, include:
- Tasks
- Acceptance criteria
- Tests
- Required deliverables
Add a final rubric-compliance checklist.
Do not implement code yet.
Do not mark incomplete tasks as completed.
```

What happened: Claude Code wrote a seven-phase TODO.md with an API plan and a grading checklist, with every box unchecked. (The original TODO had every box ticked, even though no code existed yet.) A follow-up prompt added the submission rules, such as HTTPS, prompts in the report, and the demo video.

Prompt 2 — Game logic, word lists, and API (Phases 1–3)

```
Implement Phases 1, 2, and 3 from TODO.md sequentially.
[...]
- Complete and test each phase before starting the next.
- Keep the game engine independent of Flask.
- Use in-memory game storage with unique IDs.
- Validate inputs and return appropriate JSON errors.
- Never expose hidden words during active games.
- Create comprehensive pytest tests for each phase.
[...]
- Update TODO.md only for verified completed tasks.
```

What happened: it created *words.py*, *game.py*, and *server.py* with 90 passing tests, and ran a live check of all three endpoints. It left the main-page item unchecked until the real page was built in Phase 4.

Prompt 3 — Web page and testing (Phases 4–5)

```
Implement Phases 4 and 5 from TODO.md sequentially.
[...]
- Draw all six Hangman stages.
[...]
- Support New Game without refreshing.
- Use fetch() with the existing Flask API.
- Keep the hidden word server-side.
- Handle network and API errors.
[...]
- Do not claim manual browser tests passed unless performed.
```

What happened: it built the web page, added integration tests, ran the background Chrome checks, and left the manual checklist blank for a person to fill in.

8. What I Learned

Letting the server make every decision made hiding the word simple: one rule in one place. Writing down what "done" means before coding made each step easy to check. I also learned not to trust work just because it looks finished, even when an AI wrote it. The mistake above was only caught by comparing the checklist with the actual code.